贾志伟个人简历

18155129347 | 550844389@qq.com | 合肥 | java 开发工程师

自我评价

7 年 java 开发经验,具备独立的设计开发能力，热衷于新技术的钻研

教育经历

吉首大学 **2013 年 06 月-2017 年 6 月**

软件工程 本科 湖南

工作经历

安徽远航交通科技有限公司	2024 年 6 月-至今	合肥
掌触信息科技有限公司	2022 年 5 月-2024 年 6 月	合肥
京北方信息技术有限公司	2020 年 7 月-2022 年 4 月	合肥
安徽航天信息科技有限公司	2019 年 3 月-2020 年 6 月	合肥

专业技能

- 熟练使用 springcloud、spring、myabtis 进行开发
- 熟练使用 redis、mq、es、rocketMq、xxl-job 等中间件
- 深入理解 RocketMQ 中间件 consumer、producer 端底层原理，掌握 RocketMQ 高可用方案。

- 深入理解 mysql innodb 存储引擎缓冲池底层工作原理
- 深入理解 JVM 代码执行原理、ParNew、CMS、G1 垃圾回收原理
- 熟悉常用的设计模式

项目经验

Harness Workflow 研发流程平台

项目背景：针对智能体接入研发后出现的需求流转失控、质量门禁缺失、问题回归链路不闭环等问题，自主设计并实现一套可复用的研发流程平台，用于支撑不同项目的标准化协同开发。

项目职责：

流程引擎设计：设计 requirement/change/plan/version 多级工作模型，以及 workflow-runtime、meta 状态机，支持阶段推进、人工审核、版本切换、状态修复与旧项目升级。

平台工具建设：实现 Harness CLI、模板体系和受管文件同步机制，兼容 Codex、Claude Code 双端接入，支持初始化、更新、归档、改名等平台能力。

质量门禁建设：将 mvn compile、项目启动验证、问题回归、用户输入模板纳入统一流程，形成“发现问题-定位原因-补充数据-继续修复”的闭环。

稳定性与维护能力：实现 archive、rename、update drift repair、经验索引等机制，解决目录改名、状态漂移、历史项目升级不一致等问题。

遇到的问题：

问题描述：流程平台在实际使用中容易出现状态与目录不一致、历史项目升级后元数据漂移，以及编译失败、启动失败后责任边界不清的问题。

解决方案：通过状态机约束执行路径，结合 active/update/rename 等修复机制，以及

regression、required-inputs、经验沉淀等能力，保证研发流程可追踪、可恢复、可复用。

无人机巡检系统

- **项目背景：**
- **项目职责：**

核心业务模块实现： 独立或主导完成了定时任务、巡检/面状航线管理、飞行记录与轨迹回放、OSD 数据传输、键盘控制、负载控制等核心功能的开发。这些模块构成了无人机自动化作业的基础，确保了飞行任务的灵活配置与执行监控。

数据应用与报告生成： 参与了部分数据统计功能以及巡检报告模块的开发，将原始的飞行和巡检数据转化为有价值的分析结果和格式化报告，提升了用户的数据利用效率。

拓展功能与成果管理： 负责了部分 PSDK 喊话器功能的开发，拓展了无人机在应急、指挥等场景的应用能力。同时，完成了建模成果模块和调试功能的开发，完善了从数据采集到成果生成、查看的完整链路。

飞控厂商对接： 与嗨森、安飞等主流飞控厂商完成了具体功能接口的对接与联调，保障了无人机硬件与平台软件的顺畅通信，扩大了平台的硬件适配范围。

建模厂商合作： 与云端地球等建模厂商完成了建模功能的对接，实现了飞行数据与建模服务的无缝集成，为用户提供了一站式的解决方案。

技术能力输出： 将已有的飞控、视频流等核心能力，通过标准接口及 MQTT 等形式进行封装和暴露，成功与陕西交科、南京感动科技等合作伙伴的系统完成对接。这不仅验证了平台的技术开放性和稳定性，也为后续更多的生态合作奠定了基础。

- **遇到的问题：**

自研基于上云 API 的飞控系统

- **项目背景：**
- **项目职责：**

核心算法与任务生成： 负责实现了大疆标准航线包、斜摄影航线包、正摄影像航

线包的自动生成功能，为不同业务场景下的自动化飞行提供了核心算法支持。

全流程功能开发： 完成了自研飞控中定时任务、飞行记录、轨迹记录、飞行控制等关键功能的开发，确保了自研系统在功能上对标主流标准，具备独立作业的能力。

- **遇到的问题：**

会员中台系统

- **项目背景：**

该项目是一套 saas 系统，目的是为了解决企业私域流量维护的问题。项目目前包含了积分商城、会员中心、活动中心三大块，同时打通了与数云 crm 之间的会员数据同步，单租户会员总量最多 1700w。

- **项目职责：**

参与会员中台从 0-1 的开发，主要负责会员中心后台相关设计以及开发，涉及 PC 管理端、小程序端、以及与数云之间的三方同步。

会员中心包含了会员注册、客户资料维护、会员等级相关、会员积分相关、会员权益、积分任务、企业微信标签、会员营销（各种礼包活动）、会员信息实时统计、三方会员数据以及积分数据同步等功能。

- **遇到的问题：**

➢ **ES 深分页问题**

问题描述:

es 数据量超过 1w 条时会查询失败,本质上是因为 es 分片存储数据，限制为 1w 条是为了保护系统，防止 OOM

解决方案:

添加 serialNo 字段,针对数据量超过 1w 的分页通过当前最大数据量去计算下一页的 serialNo，并查询下于该值的数据，同时分页参数改为 1 只查询第一页。

➢ **大流量下接口过慢问题**

问题描述:

某些接口例如注册、签到等，由于 db 数据量，流程过较长或与三方交互等问题，导致接口过慢。

解决方案:

1. 数据库层面百万级别以下的表通过设计好索引，百万以上级别通过数据库的分区来解决
2. 与主流程关联性较强的操作通过多线程异步处理，与主流程关联性不大的操作可以通过 MQ 进行异步处理。

- **ES 写数据延迟所带来的问题**

问题描述:

es 数据写入会有 2s 左右的延迟，这样在执行一些要求数据实时的操作时就会存在问题，例如注册之后新人奖励的发放等

解决方案:

1. 数据量较小的表通过做好索引直接走库查询即可
2. 数据量较大的表引入 redis 缓存，redis 中查询不到再走 es 查询

- **积分数据操作如何保障准确性**

问题描述:

积分可以在系统中换各种商品，相当于金钱，所以要求发放的准确性，以及与三方系统的一致性。

解决方案:

1. 积分日志中引入是否发放标识，并且每晚会对前一天的日志进行扫描，未发放的记录进行补发。
2. 积分发放通过统一的服务进行发放，并根据用户以及积分操作类型

加 redis 锁。

3. 获取用户积分时会触发三方积分的同步，频率设置为 2 小时一次，

同步时需要进行积分记录比对，保障最新的 20 条记录与三方记录一致。

- **会员营销以及会员任务等如何应对频繁的新增需求带来的代码改动**

问题描述:

公司会不断的提一些新的营销玩法以及会员任务来提高用户活跃度，每次提供新的需求都需要单独再开发。

解决方案:

1. 抽出共有的活动配置，并新增一张自定义配置表，以应对配置通用

以及自定义

2. 通过模板方法模式与抽象工厂模式结合,提供一套公共的奖励发放并

允许自定义各种校验以及奖励逻辑

3. 暴露通用的奖励发放接口，结合自定义注解和枚举来选择不同的奖

励发放组件，这样当新增通用的营销活动或者任务时直接刷入配置即可,需要

特殊定制的组件直接继承抽象类用自定义注解来标识就可以被工厂调用。

信息科技综合管理系统

- **项目背景**：该项目的主要用途有两块

1. 将行内的零散的各个系统综合起来，达到一个全面的管理系统，

2. 开发一套针对软件开发全流程的线上管理项目。包括申报、立项、审批、采

购、业务架构管理、软件架构管理、需求管理、工程管理、工程进展追踪、工程上线管理等

- **项目职责**

参与采购管理、业务架构管理、工程管理、工程周进展跟踪、工程上线管理相关模块等开发

- **遇到的问题**
- **接口以及页面冗余问题**

问题描述:

各模块间页面存在高度相似页面以及 excel 导入导出等功能被大量重复开

发

措施：

·后端开发了一套通用等 excel 服务，可以提供 excel 查询（单表、联表查询）、导入、导出功能

·前端同步开发了一套基于 nui 的 js 脚本，可以调用 excel 服务生成通用页面，并生成一套基础的页面。

·前后端联动后只需要对数据进行配置就可以达到对页面生成，并可以很方便的对查询条件进行新增，对导入导出功能进行开放关闭。

- **新老工程管理系统同步运行工程数据缺失**

问题描述：

工程管理模块上线初期，因为要对新系统进行全国对推广，所以会存在新老系统并行使用的问题，老工程管理系统以前的数据是没有的，新增的数据也不会同步到这边的工程管理模块中。

措施：

·首先历史是数据问题，在上线当晚通过脚本将老系统的工程库数据同步到新的工程管理模块的库中

·然后就是数据变更问题，针对老工程管理系统工程库做 binlog 监听，一旦监听到表数据有变动，就通过 rocketMQ 发送一条数据，到工程管理系统，做对应数据的变更。

- **线上工程库中出现多条相同工程数据的问题**

问题描述：

线上工程库中出现了多条一样数据的问题，由于我们是敏捷开发，所以初期项目体量不大，线上部署是单节点部署，后期改为了多节点部署，这时相关人员没有考虑到定时任务多节点部署时多次执行的问题，由工程库同步过来的数据被多次入库。

措施：

用 redis 的 setNx 做分布式锁，并给每个定时任务给到一个不同的 key，这样既能保证不会重复执行，也能保证充分利用多节点部署的优势最大化使用机器性能。